# Exercises

## Exercise 9: Version control with Git and GitHub

Read Chapter 9 to help you complete the questions in this exercise.

**Before the session.** This exercise assumes Git is installed, you have a GitHub account, and RStudio can authenticate with it. That takes about half an hour and is fiddly the first time, so please work through the **Setting up Git and GitHub** page before you arrive. If you hit a real problem, bring it to the start of the session and we will help you, but we cannot get everybody set up and still teach anything, so please try first.

So far, your safety net has been saving your script and hoping. Version control replaces that with something better: a record of every version of your work, who changed what and when, and why. It is the difference between a folder full of `analysis_final_v2_REALLY_final.R` and a project with a history you can actually read.

Git is the program that keeps that history on your computer. GitHub is a website that keeps a copy, which makes it a backup, a way to share work, and somewhere to point people who ask what you have been doing.

We will only cover the everyday loop in this exercise. That loop is what you will use ninety-nine times out of a hundred.

1. Start by checking your setup is healthy. In the R console run the situation report:

```
usethis::git_sitrep()
```

This prints a lot at once, which is normal. You are looking for just two things: your name and email address near the top, under the Git config heading, and a line further down confirming that a personal access token was found. Everything else you can ignore for now.

If either of those is missing, go back to the setup page and work out which step did not take. Ask for help now rather than at Q4, when the symptom will be an error message rather than a missing line.

2. Now create somewhere for your work to live. We are going to make the repository on GitHub first and then bring it down to your computer, because that way the two ends are connected from the start and there is less to go wrong.

a) Go to github.com and log in. Click the + in the top right and choose **New repository**.

b) Name it `pu5058-practical`. Leave it **Public**. Tick **Add a README file**. From the **Add .gitignore** dropdown, choose the **R** template. Click **Create repository**.

c) On the page that appears, click the green **Code** button and copy the HTTPS web address. It will end in `.git`.

d) In RStudio, go to `File -> New Project...`. Choose **Version Control**, then **Git**. Paste the address into the 'Repository URL' box. The 'Project directory name' box fills in on its own. Under 'Create project as a subdirectory of', browse to wherever you keep your work for this course, then click **Create Project**.

RStudio restarts and opens a new Project containing the README and the `.gitignore` that GitHub made for you. You now have the same repository in two places: one on your computer and one on GitHub. Keeping those two in step is what the rest of this exercise is about.

Note this is a *different* Project from the one you have used for Exercises 1 to 7. That is deliberate, so you are not putting the whole course under version control today.

3. Look before you touch anything. Find the **Git** tab in the top right pane of RStudio (it appears only in a Project that is under version control, which is why Q2 came first). Identify each of the following, without clicking yet:

- the **Staged** column of tick boxes
- the coloured status icons and what they mean: a yellow `?` for a file Git has never seen, a blue `M` for a file that has been modified, a green `A` for a file you have staged
- the **Diff** button, which shows what changed
- the **Commit** button
- the **Pull** and **Push** buttons, the down and up arrows
- the **History** button, the little clock

4. Time for the full cycle, once, slowly. First you need something worth saving.

a) Using your computer's file manager (Finder on a Mac, File Explorer on Windows), make a `data` folder inside your new Project folder and copy `cardiacdata.txt` into it. You are copying it because this is a different Project from the one you have been using.

b) In RStudio, create a new R script (`File -> New File -> R Script`) and save it as `cardiac_analysis.R`. In it, import the data with `read.table()` as you did in **Exercise 3, Q5**, and draw the boxplot of total cholesterol by smoking status from **Exercise 5, Q10**. Do not forget the factor recoding from **Exercise 5, Q4**. Save the file.

c) Look at the Git pane. `cardiac_analysis.R` has appeared with a yellow `?` beside it, meaning Git can see it but is not yet keeping track of it. Tick the **Staged** box next to it. The icon changes to a green `A`, for 'added'.

d) Click **Commit**. A window opens showing the file you staged and, underneath, every line you are about to record. Type a message in the box on the right. Make it say *why* rather than *what*: "add script to plot cholesterol by smoking status" is a useful message, "update" is not, and in six months you will be grateful. Click the **Commit** button, read the confirmation, and close it.

e) Your change is now saved in the history on your computer, but GitHub knows nothing about it. Click the green **Push** arrow. Wait for it to finish, then go to your repository page on GitHub in your browser and refresh. Your script is there.

If the push fails with a message about authentication, your token is the problem rather than anything you have done here. See the last section of the setup page.

5. Once is a demonstration, twice is a habit. Open `cardiac_analysis.R`, change something small (adding a title to your plot with the `main` argument will do), and save it. The file reappears in the Git pane, this time with a blue `M` for 'modified'.

a) Before you commit, select the file and click **Diff**. Read what it shows you: added lines in green, removed lines in red. Getting into the habit of looking at the diff before committing catches an enormous number of mistakes.

b) Stage, commit with a sensible message, and push.

c) Now click **History**, the clock icon. You should see both of your commits, most recent first, with their messages, authors and dates. Click on the first commit to see exactly what it contained. This is the record that makes version control worth the trouble: not the backup, but being able to see what past you did and what past you was thinking.

6. Everyone makes changes they wish they had not. Git gives you a way back.

a) Open your script, delete the whole line that draws the boxplot, and save it. Do **not** commit. Now, in the Git pane, right-click on the file and choose **Revert**. Say yes to the warning, then look at your script again: the deleted line is back.

Be careful with this one. It is not an undo button. It throws away *everything* you have changed since your last commit, not just the last thing you did, and there is no way back. Commit often and you will rarely need it.

b) Now a subtler one. Commit a change, but **do not push it**. Realise you forgot to include something, add it, and then in the Commit window tick **Amend previous commit** before committing again. Rather than making a second commit, this folds your new change into the previous one. This is only safe before you push. Once a commit is on GitHub, other people may have it, and rewriting it causes them problems.

7. So far everything has flowed from your computer to GitHub. It flows the other way too, and this is the half people forget.

a) In your browser, open your repository on GitHub, click on `README.md`, and click the pencil icon to edit it. Write a couple of sentences describing what the project is for. Scroll down and click **Commit changes**.

b) Your computer knows nothing about this. In RStudio, click **Pull**, the blue down arrow. Open the README in RStudio and your change is there.

c) Now think about why this matters. If you had edited the README on your computer as well before pulling, Git would have had two different versions of the same file and would have asked you to sort it out. That is a merge conflict, and it is why the advice is always **pull before you push**. Get into the habit now, while you are working alone and there is nothing to conflict with.

8. Finally, what not to put in a repository. Open the `.gitignore` file in your Project. This is the list of things Git deliberately ignores, and GitHub gave you a sensible starting one when you chose the R template in Q2.

a) Read through it. You should recognise `.Rhistory` and `.RData`.

b) Add a line ignoring anything in an `output` directory, since generated files can always be regenerated and do not need a history.

c) Right-click a file in the Git pane and notice there is an **Ignore** option, which adds it to `.gitignore` for you.

d) Think about your data. `cardiacdata.txt` is small, unchanging and not confidential, so committing it is reasonable and makes the project self-contained. Real patient data would be a different matter entirely, and the rule there is simple: **never put confidential data or passwords in a repository**, and be especially careful with public ones, because deleting a file does not remove it from the history.

**Where to go next**

Git can do a great deal more than this. **Branches** let you work on something risky without disturbing the version that works. **Forks** and **pull requests** are how people contribute to each other's projects. **Merge conflicts**, which you will meet the first time you and somebody else edit the same file, have their own set of tools. None of that is needed for this course, and all of it is easier to learn once the everyday loop in this exercise is second nature.

When you want to go further, the standard reference for R users is Happy Git and GitHub for the useR, which is free online. Section 9.6.4 of the Introduction to R book gives an overview of how collaboration works.

One last thing worth knowing: the website you are reading this on is built from a public GitHub repository, using exactly the tools in this exercise. The **Source code** link in the menu at the top of this page will take you to it.

End of Exercise 9